Explainable IDS — Full Pipeline

ICCN-INE2 Project 5 | NSL-KDD | MLP + LSTM + 1D-CNN | SHAP + LIME

0. Setup

```
!pip install -q torch numpy pandas scikit-learn datasets shap lime matplotlib scipy
```

```
import os, sys, json, time, random, pickle
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
from torch.utils.data import TensorDataset, DataLoader
from sklearn.preprocessing import LabelEncoder, MinMaxScaler
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score, average_precision_score
from datasets import load_dataset
import shap
from lime import lime_tabular
from scipy.stats import spearmanr, pearsonr
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings('ignore')

# Reproducibility
SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
torch.backends.cudnn.deterministic = True
torch.backends.cudnn.benchmark = False

DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'Device: {DEVICE}')
if DEVICE.type == 'cuda':
    print(f'GPU: {torch.cuda.get_device_name(0)}')
```

```
Device: cuda
GPU: Tesla T4
```

1. Load & Preprocess NSL-KDD

```
FEATURE_NAMES = [
    'duration', 'protocol_type', 'service', 'flag',
    'src_bytes', 'dst_bytes', 'land', 'wrong_fragment', 'urgent',
    'hot', 'num_failed_logins', 'logged_in', 'num_compromised',
    'root_shell', 'su_attempted', 'num_root', 'num_file_creations',
    'num_shells', 'num_access_files', 'num_outbound_cmds',
    'is_host_login', 'is_guest_login',
    'count', 'srv_count',
    'serror_rate', 'srv_serror_rate', 'rerror_rate', 'srv_rerror_rate',
    'same_srv_rate', 'diff_srv_rate', 'srv_diff_host_rate',
    'dst_host_count', 'dst_host_srv_count',
    'dst_host_same_srv_rate', 'dst_host_diff_srv_rate',
    'dst_host_same_src_port_rate', 'dst_host_srv_diff_host_rate',
    'dst_host_serror_rate', 'dst_host_srv_serror_rate',
    'dst_host_rerror_rate', 'dst_host_srv_rerror_rate'
]
CATEGORICAL_COLS = ['protocol_type', 'service', 'flag']

# Load from HuggingFace
ds = load_dataset('Mireu-Lab/NSL-KDD')
df_train = ds['train'].to_pandas()
df_test = ds['test'].to_pandas()
print(f'Train: {len(df_train)} | Test: {len(df_test)}')

# Class distribution
```

```
print('\nTrain distribution:')
print(df_train['class'].value_counts())
print('\nTest distribution:')
print(df_test['class'].value_counts())
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits a
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_
README.md:        2.28k/?  [00:00<00:00,  163kB/s]

train.csv:  100%                                                    22.5M/22.5M  [00:01<00:00,  112MB/s]

test.csv:        5.14M/?  [00:00<00:00,  27.6MB/s]

Generating  train  split:  100%                                                    151165/151165  [00:01<00:00,  152887.76  examples/s]

Generating  test  split:  100%                                                    34394/34394  [00:00<00:00,  131820.95  examples/s]
Train: 151165 | Test: 34394

Train distribution:
class
normal     80792
anomaly    70373
Name: count, dtype: int64

Test distribution:
class
anomaly    22531
normal     11863
Name: count, dtype: int64
```

```
# Encode target (binary: anomaly=0, normal=1)
class_names = ['anomaly', 'normal']
le_y = LabelEncoder()
y_train = le_y.fit_transform(df_train['class'].values)
y_test = le_y.transform(df_test['class'].values)

# Encode categoricals
df_tr, df_te = df_train.copy(), df_test.copy()
label_encoders = {}
for col in CATEGORICAL_COLS:
    le = LabelEncoder()
    le.fit(df_tr[col])
    known = set(le.classes_)
    df_te[col] = df_te[col].apply(lambda x: x if x in known else le.classes_[0])
    df_tr[col] = le.transform(df_tr[col])
    df_te[col] = le.transform(df_te[col])
    label_encoders[col] = le
    print(f'Encoded {col}: {len(le.classes_)} categories')

# Scale features
scaler = MinMaxScaler()
X_train = scaler.fit_transform(df_tr[FEATURE_NAMES].values.astype(np.float32))
X_test = scaler.transform(df_te[FEATURE_NAMES].values.astype(np.float32))

print(f'\nX_train: {X_train.shape} | X_test: {X_test.shape}')
print(f'y_train: {np.bincount(y_train)} | y_test: {np.bincount(y_test)}')
```

```
Encoded protocol_type: 3 categories
Encoded service: 70 categories
Encoded flag: 11 categories

X_train: (151165, 41) | X_test: (34394, 41)
y_train: [70373 80792] | y_test: [22531 11863]
```

2. Model Definitions

```
class MLP_IDS(nn.Module):
    def __init__(self, in_dim=41, num_classes=2):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(in_dim, 256), nn.BatchNorm1d(256), nn.ReLU(), nn.Dropout(0.3),
            nn.Linear(256, 128), nn.BatchNorm1d(128), nn.ReLU(), nn.Dropout(0.2),
            nn.Linear(128, 64), nn.ReLU(),
            nn.Linear(64, num_classes)
        )
        for m in self.modules():
```

```
            if isinstance(m, nn.Linear):
                nn.init.xavier_uniform_(m.weight)
                nn.init.zeros_(m.bias)
    def forward(self, x): return self.net(x)
    def count_parameters(self): return sum(p.numel() for p in self.parameters() if p.requires_grad)

class LSTM_IDS(nn.Module):
    def __init__(self, in_dim=41, hidden_dim=64, num_layers=2, num_classes=2):
        super().__init__()
        self.lstm = nn.LSTM(1, hidden_dim, num_layers, batch_first=True, dropout=0.2)
        self.fc = nn.Sequential(nn.Linear(hidden_dim, 32), nn.ReLU(), nn.Linear(32, num_classes))
    def forward(self, x):
        out, (h_n, _) = self.lstm(x.unsqueeze(-1))
        return self.fc(h_n[-1])
    def count_parameters(self): return sum(p.numel() for p in self.parameters() if p.requires_grad)

class CNN1D_IDS(nn.Module):
    def __init__(self, in_dim=41, num_classes=2):
        super().__init__()
        self.conv = nn.Sequential(
            nn.Conv1d(1, 64, 3, padding=1), nn.BatchNorm1d(64), nn.ReLU(),
            nn.Conv1d(64, 128, 3, padding=1), nn.BatchNorm1d(128), nn.ReLU(),
            nn.AdaptiveAvgPool1d(8)
        )
        self.fc = nn.Sequential(nn.Linear(128*8, 64), nn.ReLU(), nn.Dropout(0.2), nn.Linear(64, num_classes))
    def forward(self, x):
        x = self.conv(x.unsqueeze(1))
        return self.fc(x.view(x.size(0), -1))
    def count_parameters(self): return sum(p.numel() for p in self.parameters() if p.requires_grad)

for name, cls in [('MLP', MLP_IDS), ('LSTM', LSTM_IDS), ('CNN1D', CNN1D_IDS)]:
    m = cls()
    print(f'{name}: {m.count_parameters():,} parameters')
```

```
MLP: 52,802 parameters
LSTM: 52,578 parameters
CNN1D: 91,074 parameters
```

3. Train All Models

```
EPOCHS = 50
BATCH_SIZE = 256
LR = 1e-3

# Data loaders
train_ds = TensorDataset(torch.FloatTensor(X_train), torch.LongTensor(y_train))
test_ds = TensorDataset(torch.FloatTensor(X_test), torch.LongTensor(y_test))
train_loader = DataLoader(train_ds, batch_size=BATCH_SIZE, shuffle=True)
test_loader = DataLoader(test_ds, batch_size=BATCH_SIZE)

# Class weights
counts = np.bincount(y_train)
weights = 1.0 / counts.astype(np.float32)
weights = weights / weights.sum() * len(weights)
class_weights = torch.FloatTensor(weights).to(DEVICE)

def train_model(model, model_name):
    print(f'\n{"="*60}')
    print(f'Training {model_name} ({model.count_parameters():,} params) on {DEVICE}')
    print(f'{"="*60}')

    model.to(DEVICE)
    criterion = nn.CrossEntropyLoss(weight=class_weights)
    optimizer = torch.optim.Adam(model.parameters(), lr=LR, weight_decay=1e-4)
    scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(optimizer, patience=5, factor=0.5)

    best_f1, history = 0, {'train_loss': [], 'test_acc': []}
    best_state = None
    t0 = time.time()

    for epoch in range(EPOCHS):
        model.train()
        total_loss = 0
        for xb, yb in train_loader:
            xb, yb = xb.to(DEVICE), yb.to(DEVICE)
```

```
            optimizer.zero_grad()
            loss = criterion(model(xb), yb)
            loss.backward()
            optimizer.step()
            total_loss += loss.item() * len(yb)

        # Evaluate
        model.eval()
        preds, probs, labels = [], [], []
        with torch.no_grad():
            for xb, yb in test_loader:
                xb = xb.to(DEVICE)
                out = model(xb)
                preds.append(out.argmax(1).cpu().numpy())
                probs.append(torch.softmax(out, 1).cpu().numpy())
                labels.append(yb.numpy())
        preds = np.concatenate(preds)
        probs = np.concatenate(probs)
        labels = np.concatenate(labels)

        report = classification_report(labels, preds, output_dict=True)
        wf1 = report['weighted avg']['f1-score']
        acc = report['accuracy']
        test_loss = total_loss / len(y_train)
        scheduler.step(test_loss)

        history['train_loss'].append(total_loss / len(y_train))
        history['test_acc'].append(acc)

        if wf1 > best_f1:
            best_f1 = wf1
            best_state = {k: v.cpu().clone() for k, v in model.state_dict().items()}

        if (epoch+1) % 10 == 0 or epoch == 0:
            print(f'  Epoch {epoch+1:3d}/{EPOCHS} | Loss: {total_loss/len(y_train):.4f} | Acc: {acc:.4f} | F1: {wf1:

    dt = time.time() - t0

    # Load best and final eval
    model.load_state_dict(best_state)
    model.eval()
    preds, probs, labels = [], [], []
    with torch.no_grad():
        for xb, yb in test_loader:
            xb = xb.to(DEVICE)
            out = model(xb)
            preds.append(out.argmax(1).cpu().numpy())
            probs.append(torch.softmax(out, 1).cpu().numpy())
            labels.append(yb.numpy())
    preds = np.concatenate(preds)
    probs = np.concatenate(probs)
    labels = np.concatenate(labels)

    roc = roc_auc_score(labels, probs[:, 1])
    pr = average_precision_score(labels, probs[:, 1])

    print(f'\n  Time: {dt:.1f}s | Best F1: {best_f1:.4f} | ROC-AUC: {roc:.4f} | PR-AUC: {pr:.4f}')
    print(classification_report(labels, preds, target_names=class_names))
    print('Confusion Matrix:')
    print(confusion_matrix(labels, preds))

    return model, {'f1': best_f1, 'roc_auc': roc, 'pr_auc': pr, 'time': dt, 'history': history, 'preds': preds, 'prc

# Train all 3
models = {}
results = {}
for name, cls in [('mlp', MLP_IDS), ('lstm', LSTM_IDS), ('cnn1d', CNN1D_IDS)]:
    models[name], results[name] = train_model(cls(), name.upper())
```

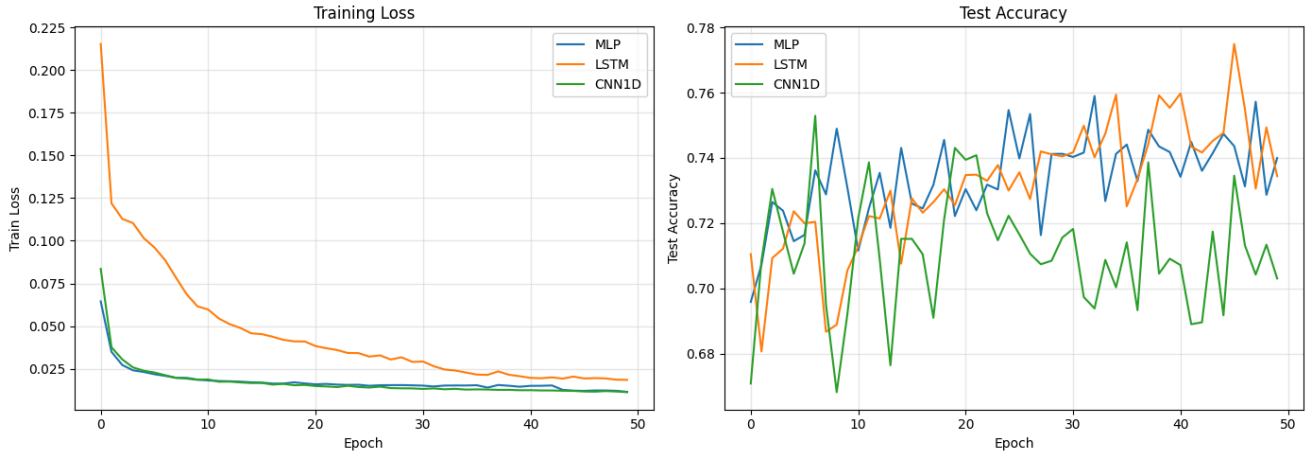
```
============================================================
Training MLP (52,802 params) on cuda
============================================================
  Epoch   1/50 | Loss: 0.0645 | Acc: 0.6959 | F1: 0.6986
  Epoch  10/50 | Loss: 0.0187 | Acc: 0.7308 | F1: 0.7353
  Epoch  20/50 | Loss: 0.0164 | Acc: 0.7222 | F1: 0.7263
  Epoch  30/50 | Loss: 0.0153 | Acc: 0.7413 | F1: 0.7459
  Epoch  40/50 | Loss: 0.0145 | Acc: 0.7418 | F1: 0.7465
```

```
  Epoch  50/50 | Loss: 0.0115 | Acc: 0.7400 | F1: 0.7447

  Time: 145.1s | Best F1: 0.7639 | ROC-AUC: 0.9231 | PR-AUC: 0.8699
              precision    recall  f1-score   support

     anomaly       0.97      0.65      0.78     22531
      normal       0.59      0.96      0.73     11863

    accuracy                           0.76     34394
   macro avg       0.78      0.81      0.76     34394
weighted avg       0.84      0.76      0.76     34394

Confusion Matrix:
[[14679  7852]
 [  438 11425]]

============================================================
Training LSTM (52,578 params) on cuda
============================================================
  Epoch   1/50 | Loss: 0.2152 | Acc: 0.7105 | F1: 0.7168
  Epoch  10/50 | Loss: 0.0616 | Acc: 0.7056 | F1: 0.7089
  Epoch  20/50 | Loss: 0.0410 | Acc: 0.7255 | F1: 0.7296
  Epoch  30/50 | Loss: 0.0290 | Acc: 0.7405 | F1: 0.7452
  Epoch  40/50 | Loss: 0.0206 | Acc: 0.7554 | F1: 0.7607
  Epoch  50/50 | Loss: 0.0185 | Acc: 0.7344 | F1: 0.7388

  Time: 162.9s | Best F1: 0.7800 | ROC-AUC: 0.9434 | PR-AUC: 0.9222
              precision    recall  f1-score   support

     anomaly       0.97      0.68      0.80     22531
      normal       0.61      0.96      0.75     11863

    accuracy                           0.77     34394
   macro avg       0.79      0.82      0.77     34394
weighted avg       0.85      0.77      0.78     34394

Confusion Matrix:
[[15263  7268]
 [  474 11389]]

============================================================
Training CNN1D (91,074 params) on cuda
============================================================
  Epoch   1/50 | Loss: 0.0835 | Acc: 0.6709 | F1: 0.6716
  Epoch  10/50 | Loss: 0.0185 | Acc: 0.6923 | F1: 0.6949
  Epoch  20/50 | Loss: 0.0156 | Acc: 0.7431 | F1: 0.7480
  Epoch  30/50 | Loss: 0.0135 | Acc: 0.7156 | F1: 0.7198
  Epoch  40/50 | Loss: 0.0124 | Acc: 0.7091 | F1: 0.7149
  Epoch  50/50 | Loss: 0.0113 | Acc: 0.7031 | F1: 0.7064
```

```
# Summary table
print(f'{"Model":<8} {"Params":>8} {"W-F1":>8} {"ROC-AUC":>9} {"PR-AUC":>8} {"Time":>8}')
print('-'*50)
for name in ['mlp', 'lstm', 'cnn1d']:
    r = results[name]
    p = models[name].count_parameters()
    print(f'{name:<8} {p:>8,} {r["f1"]:>8.4f} {r["roc_auc"]:>9.4f} {r["pr_auc"]:>8.4f} {r["time"]:>7.1f}s')
```

```
Model      Params     W-F1   ROC-AUC   PR-AUC     Time
--------------------------------------------------
mlp        52,802   0.7639    0.9231   0.8699   145.1s
lstm       52,578   0.7800    0.9434   0.9222   162.9s
cnn1d      91,074   0.7579    0.9410   0.9182   173.1s
```

```
# Training curves
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
for name in ['mlp', 'lstm', 'cnn1d']:
    axes[0].plot(results[name]['history']['train_loss'], label=name.upper())
    axes[1].plot(results[name]['history']['test_acc'], label=name.upper())
axes[0].set_xlabel('Epoch'); axes[0].set_ylabel('Train Loss'); axes[0].set_title('Training Loss'); axes[0].legend();
axes[1].set_xlabel('Epoch'); axes[1].set_ylabel('Test Accuracy'); axes[1].set_title('Test Accuracy'); axes[1].legenc
plt.tight_layout(); plt.show()
```



4. SHAP Explainability Analysis

```
# Move MLP to CPU for SHAP
mlp_cpu = models['mlp'].cpu().eval()

def predict_fn(X):
    with torch.no_grad():
        return torch.softmax(mlp_cpu(torch.FloatTensor(X)), 1).numpy()

# Background & samples
bg_idx = np.random.choice(len(X_train), 100, replace=False)
exp_idx = np.random.choice(len(X_test), 150, replace=False)

explainer = shap.KernelExplainer(predict_fn, X_train[bg_idx])
print('Computing SHAP values for 150 test samples (this takes a few minutes)...')
shap_values_raw = explainer.shap_values(X_test[exp_idx], nsamples=200, silent=True)

# Handle both old and new SHAP formats
if isinstance(shap_values_raw, list):
    shap_vals_anomaly = shap_values_raw[0]
elif shap_values_raw.ndim == 3:
    shap_vals_anomaly = shap_values_raw[:, :, 0]
else:
    shap_vals_anomaly = shap_values_raw

print(f'Done! Shape: {shap_vals_anomaly.shape}')
```

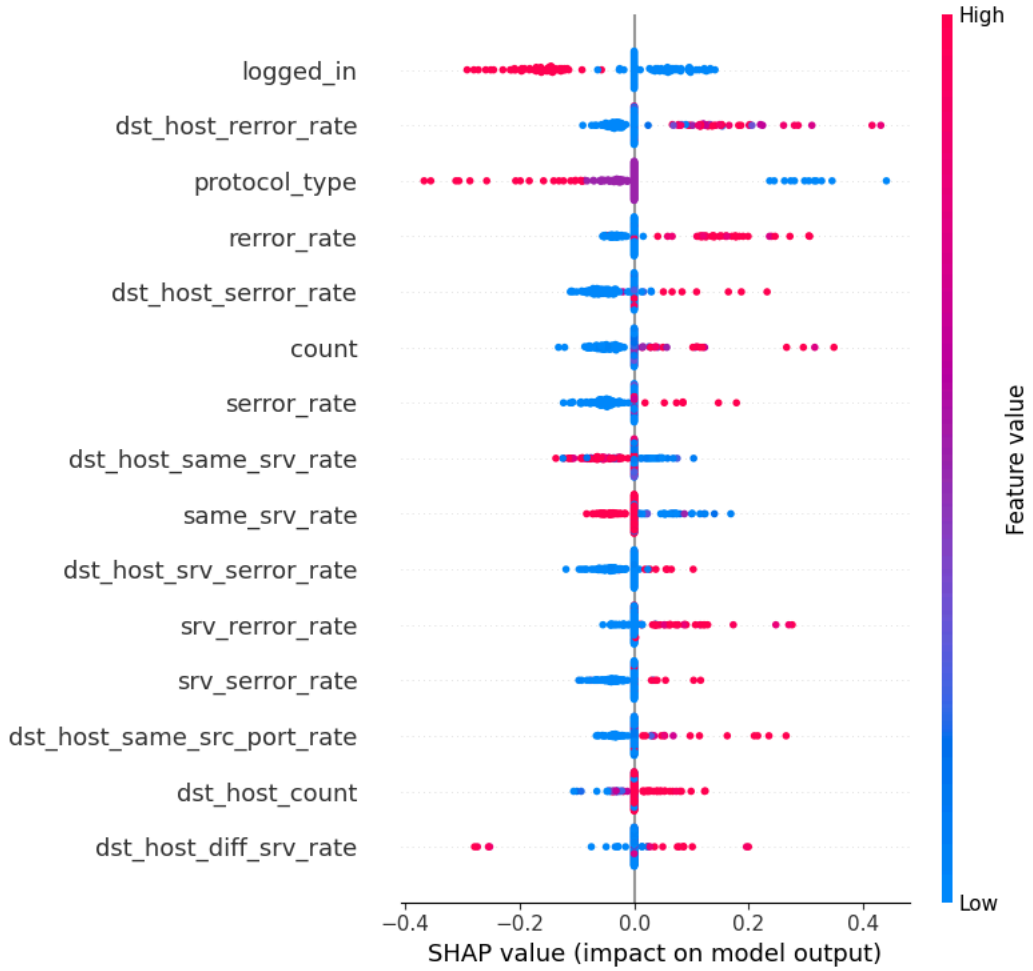
```
Computing SHAP values for 150 test samples (this takes a few minutes)...
Done! Shape: (150, 41)
```

```
# Global feature importance (anomaly class)
mean_abs_shap = np.abs(shap_vals_anomaly).mean(axis=0)
feature_importance = sorted(zip(FEATURE_NAMES, mean_abs_shap), key=lambda x: x[1], reverse=True)

print('Top 15 features by mean |SHAP| (anomaly class):')
for i, (f, v) in enumerate(feature_importance[:15]):
    print(f'  {i+1:2d}. {f:35s} {v:.4f}')
```

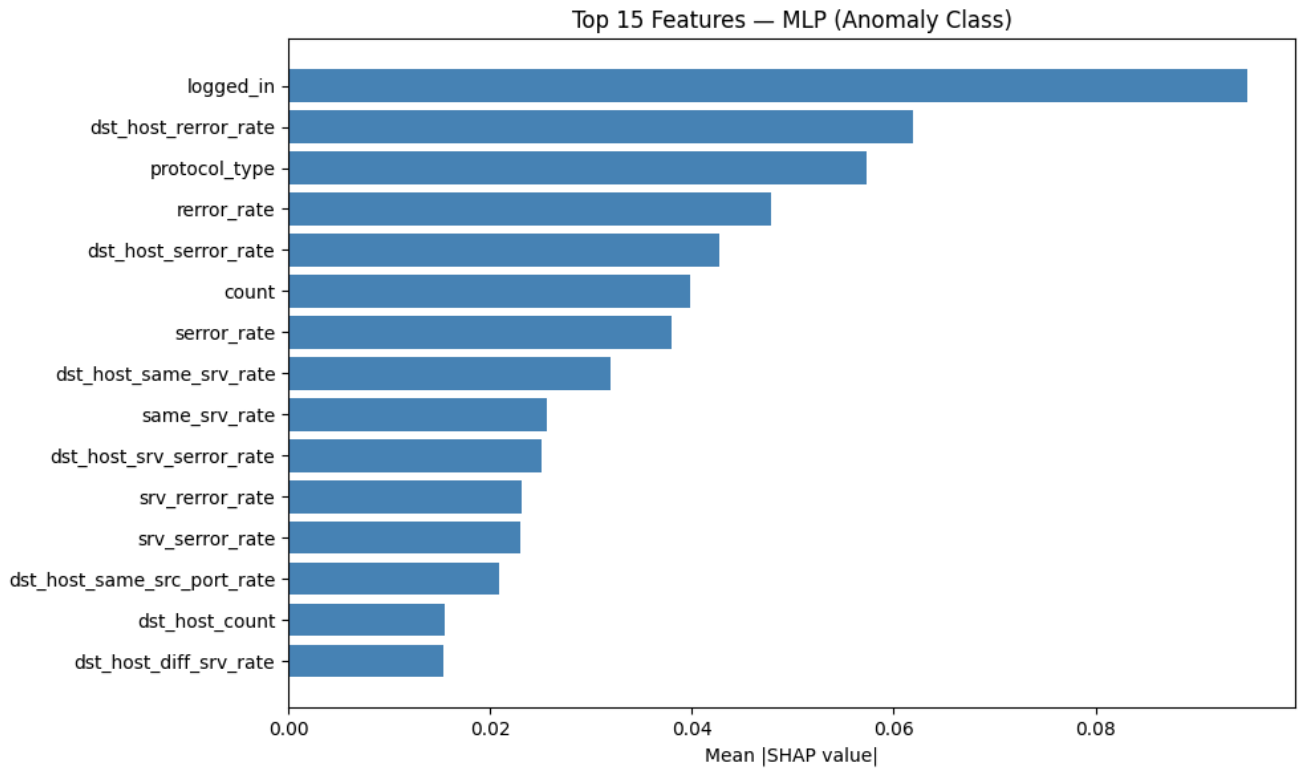
```
Top 15 features by mean |SHAP| (anomaly class):
   1. logged_in                           0.0950
   2. dst_host_rerror_rate                0.0619
   3. protocol_type                       0.0573
   4. rerror_rate                         0.0479
   5. dst_host_serror_rate                0.0427
   6. count                               0.0398
   7. serror_rate                         0.0380
```

```
  8. dst_host_same_srv_rate              0.0319
  9. same_srv_rate                       0.0256
 10. dst_host_srv_serror_rate            0.0252
 11. srv_rerror_rate                     0.0231
 12. srv_serror_rate                     0.0230
 13. dst_host_same_src_port_rate         0.0209
 14. dst_host_count                      0.0156
 15. dst_host_diff_srv_rate              0.0153
```

```
# SHAP summary plot
shap.summary_plot(shap_vals_anomaly, X_test[exp_idx], feature_names=FEATURE_NAMES, max_display=15)
```

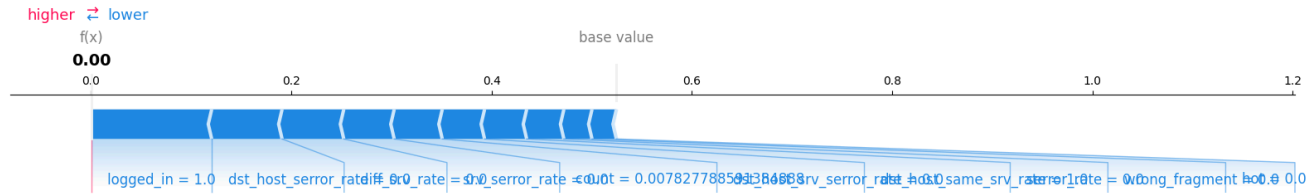



```
# SHAP bar plot
plt.figure(figsize=(10, 6))
top15 = feature_importance[:15]
plt.barh(range(15), [v for _, v in top15][::-1], color='steelblue')
plt.yticks(range(15), [f for f, _ in top15][::-1])
plt.xlabel('Mean |SHAP value|')
plt.title('Top 15 Features — MLP (Anomaly Class)')
plt.tight_layout(); plt.show()
```




```
# Single prediction explanation (force plot)
idx = 0
pred = predict_fn(X_test[exp_idx[idx:idx+1]])
print(f'Sample prediction: anomaly={pred[0][0]:.3f}, normal={pred[0][1]:.3f}')
print(f'True label: {class_names[y_test[exp_idx[idx]]]}')

ev = explainer.expected_value
ev0 = ev[0] if isinstance(ev, (list, np.ndarray)) else ev
shap.force_plot(ev0, shap_vals_anomaly[idx], X_test[exp_idx[idx]], feature_names=FEATURE_NAMES, matplotlib=True)
```

```
Sample prediction: anomaly=0.000, normal=1.000
True label: normal
```



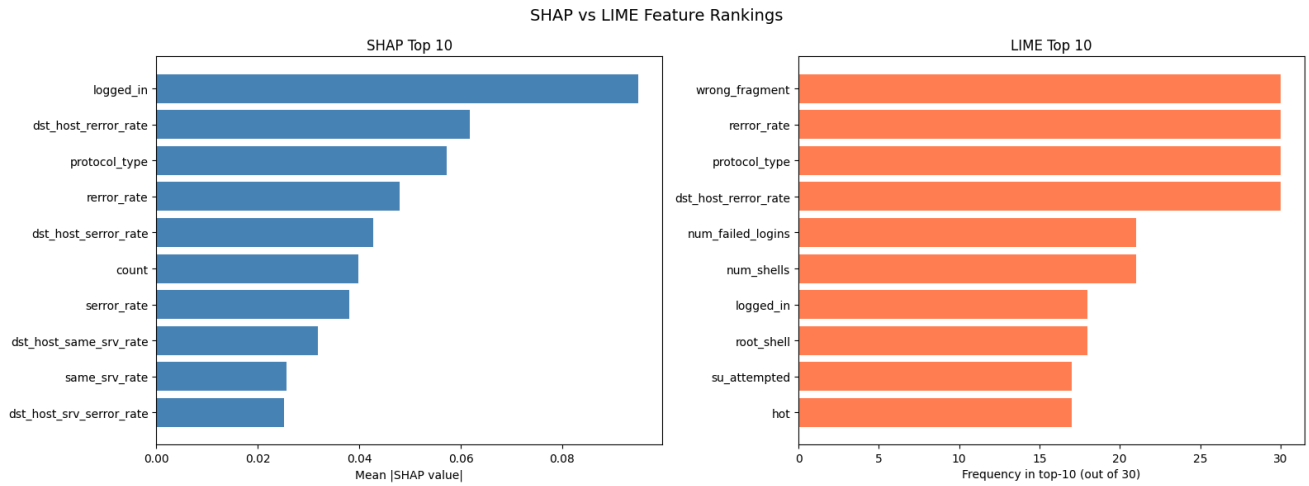
5. LIME Analysis

```
lime_explainer = lime_tabular.LimeTabularExplainer(
    X_train, feature_names=FEATURE_NAMES, class_names=class_names,
    discretize_continuous=True, random_state=SEED
)

n_lime = 30
```

```
lime_idx = np.random.choice(len(X_test), n_lime, replace=False)
all_top_features = {}

print(f'Running LIME on {n_lime} samples...')
for i, idx in enumerate(lime_idx):
    exp = lime_explainer.explain_instance(X_test[idx], predict_fn, num_features=10, top_labels=1)
    pred_class = np.argmax(predict_fn(X_test[idx].reshape(1, -1)))
    for fw in exp.as_list(label=pred_class):
        fname = fw[0].split(' ')[0]
        all_top_features[fname] = all_top_features.get(fname, 0) + 1
    if (i+1) % 10 == 0:
        print(f'  {i+1}/{n_lime} done')

lime_sorted = sorted(all_top_features.items(), key=lambda x: x[1], reverse=True)
print(f'\nTop 10 features by LIME frequency:')
for f, c in lime_sorted[:10]:
    print(f'  {f:35s}: {c}/{n_lime} explanations')
```

```
Running LIME on 30 samples...
  10/30 done
  20/30 done
  30/30 done

Top 10 features by LIME frequency:
  wrong_fragment                     : 30/30 explanations
  rerror_rate                        : 30/30 explanations
  protocol_type                      : 30/30 explanations
  dst_host_rerror_rate               : 30/30 explanations
  num_failed_logins                  : 21/30 explanations
  num_shells                         : 21/30 explanations
  logged_in                          : 18/30 explanations
  root_shell                         : 18/30 explanations
  su_attempted                       : 17/30 explanations
  hot                                : 17/30 explanations
```

```
# LIME vs SHAP comparison
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# SHAP
top10_shap = feature_importance[:10]
axes[0].barh(range(10), [v for _, v in top10_shap][::-1], color='steelblue')
axes[0].set_yticks(range(10)); axes[0].set_yticklabels([f for f, _ in top10_shap][::-1])
axes[0].set_xlabel('Mean |SHAP value|'); axes[0].set_title('SHAP Top 10')

# LIME
top10_lime = lime_sorted[:10]
axes[1].barh(range(10), [v for _, v in top10_lime][::-1], color='coral')
axes[1].set_yticks(range(10)); axes[1].set_yticklabels([f for f, _ in top10_lime][::-1])
axes[1].set_xlabel(f'Frequency in top-10 (out of {n_lime})'); axes[1].set_title('LIME Top 10')

plt.suptitle('SHAP vs LIME Feature Rankings', fontsize=14)
plt.tight_layout(); plt.show()

# Rank correlation
shap_ranks = {f: i for i, (f, _) in enumerate(feature_importance[:20])}
lime_ranks = {f: i for i, (f, _) in enumerate(lime_sorted[:20])}
common = set(shap_ranks.keys()) & set(lime_ranks.keys())
if len(common) >= 5:
    rho, p = spearmanr([shap_ranks[f] for f in common], [lime_ranks[f] for f in common])
    print(f'\nSHAP vs LIME Spearman correlation: {rho:.4f} (p={p:.4f}) over {len(common)} common features')
```




```
SHAP vs LIME Spearman correlation: 0.0714 (p=0.8665) over 8 common features
```

6. Explanation Stability Evaluation

```
def compute_shap_stability(explainer, sample, epsilon, n_perturbs=10):
    """Compute SENS_MAX and PCC for one sample."""
    rng = np.random.RandomState(SEED)
    base = np.array(explainer.shap_values(sample.reshape(1,-1), nsamples=100, silent=True))
    base = base[0].flatten() if isinstance(base, list) else base.flatten()

    max_delta, pccs = 0, []
    for _ in range(n_perturbs):
        noise = rng.uniform(-epsilon, epsilon, sample.shape)
        perturbed = np.clip(sample + noise, 0, 1)
        p_shap = np.array(explainer.shap_values(perturbed.reshape(1,-1), nsamples=100, silent=True))
        p_shap = p_shap[0].flatten() if isinstance(p_shap, list) else p_shap.flatten()
        max_delta = max(max_delta, np.linalg.norm(p_shap - base))
        if np.std(base) > 1e-8 and np.std(p_shap) > 1e-8:
            pccs.append(pearsonr(base, p_shap)[0])
    return max_delta, np.mean(pccs) if pccs else 0.0

# Test across epsilon values
epsilons = [0.01, 0.03, 0.05]
n_stability = 8
stability_idx = np.random.choice(len(X_test), n_stability, replace=False)
stability_results = {}

for eps in epsilons:
    sens_list, pcc_list = [], []
    print(f'\n--- SHAP Stability (eps={eps}) ---')
    for i, idx in enumerate(stability_idx):
        sm, pc = compute_shap_stability(explainer, X_test[idx], eps, n_perturbs=8)
        sens_list.append(sm); pcc_list.append(pc)
        if (i+1) % 4 == 0:
            print(f'  {i+1}/{n_stability} | SENS_MAX={sm:.4f} | PCC={pc:.4f}')

    stability_results[eps] = {'sens_max': np.mean(sens_list), 'pcc': np.mean(pcc_list)}
    status = 'STABLE' if np.mean(pcc_list) > 0.6 else 'UNSTABLE'
    print(f'  Mean SENS_MAX={np.mean(sens_list):.4f} | Mean PCC={np.mean(pcc_list):.4f} [{status}]')
```

```
--- SHAP Stability (eps=0.01) ---
```

```
  4/8 | SENS_MAX=0.3518 | PCC=0.5026
  8/8 | SENS_MAX=0.2409 | PCC=0.7095
  Mean SENS_MAX=0.3130 | Mean PCC=0.6293 [STABLE]

--- SHAP Stability (eps=0.03) ---
  4/8 | SENS_MAX=0.3538 | PCC=0.4325
  8/8 | SENS_MAX=0.4528 | PCC=0.5971
  Mean SENS_MAX=0.3751 | Mean PCC=0.5861 [UNSTABLE]

--- SHAP Stability (eps=0.05) ---
  4/8 | SENS_MAX=0.2833 | PCC=0.5292
  8/8 | SENS_MAX=0.6806 | PCC=0.3642
  Mean SENS_MAX=0.4527 | Mean PCC=0.5676 [UNSTABLE]
```

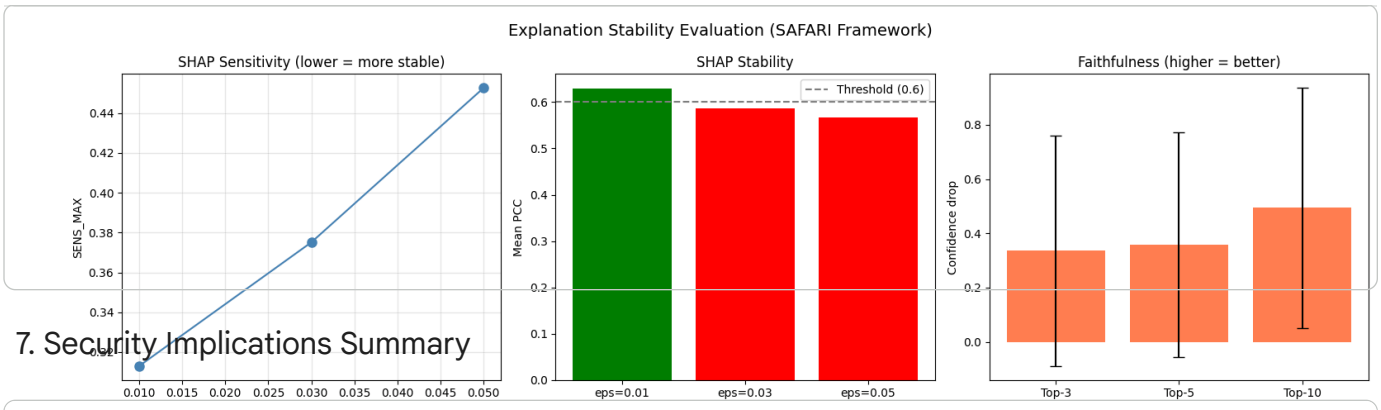
```
# LIME stochastic stability
print('--- LIME Stochastic Stability ---')
lime_corrs = []
for i, idx in enumerate(stability_idx[:6]):
    weight_vecs = []
    for seed in range(10):
        le_obj = lime_tabular.LimeTabularExplainer(X_train, feature_names=FEATURE_NAMES, discretize_continuous=True,
        exp = le_obj.explain_instance(X_test[idx], predict_fn, num_features=len(FEATURE_NAMES))
        w = np.zeros(len(FEATURE_NAMES))
        for key, val in dict(exp.as_list()).items():
            for j, fn in enumerate(FEATURE_NAMES):
                if fn in key: w[j] = val; break
        weight_vecs.append(w)
    corrs = []
    for a in range(10):
        for b in range(a+1, 10):
            if np.std(weight_vecs[a]) > 1e-8 and np.std(weight_vecs[b]) > 1e-8:
                corrs.append(spearmanr(weight_vecs[a], weight_vecs[b])[0])
    mc = np.mean(corrs) if corrs else 0
    lime_corrs.append(mc)
    print(f'  Sample {i+1}/6 | Mean Spearman: {mc:.4f}')

lime_status = 'STABLE' if np.mean(lime_corrs) > 0.6 else 'UNSTABLE'
print(f'\nOverall LIME stability: {np.mean(lime_corrs):.4f} [{lime_status}]')
```

```
--- LIME Stochastic Stability ---
  Sample 1/6 | Mean Spearman: 0.5912
  Sample 2/6 | Mean Spearman: 0.6087
  Sample 3/6 | Mean Spearman: 0.5953
  Sample 4/6 | Mean Spearman: 0.6018
  Sample 5/6 | Mean Spearman: 0.6198
  Sample 6/6 | Mean Spearman: 0.6155

Overall LIME stability: 0.6054 [STABLE]
```

```
# Faithfulness evaluation
def get_shap_for_class(shap_values, class_idx=0):
    if isinstance(shap_values, list):
        return shap_values[class_idx]
    elif isinstance(shap_values, np.ndarray) and shap_values.ndim == 3:
        return shap_values[:, :, class_idx]
    else:
        return shap_values

print('--- Faithfulness (Feature Masking) ---')
faith_results = {k: [] for k in [3, 5, 10]}

for idx in stability_idx[:10]:
    sample = X_test[idx]
    sv_raw = explainer.shap_values(sample.reshape(1,-1), nsamples=100, silent=True)
    sv = get_shap_for_class(sv_raw, 0).flatten()

    base_conf = predict_fn(sample.reshape(1,-1))[0]
    pred_cls = np.argmax(base_conf)

    for k in faith_results:
        masked = sample.copy()
        masked[np.argsort(np.abs(sv))[-k:]] = 0.0
        drop = base_conf[pred_cls] - predict_fn(masked.reshape(1,-1))[0][pred_cls]
        faith_results[k].append(float(drop))

for k, scores in faith_results.items():
    print(f'  Top-{k} masking: confidence drop = {np.mean(scores):.4f} +/- {np.std(scores):.4f}')
```

```
--- Faithfulness (Feature Masking) ---
  Top-3 masking: confidence drop = 0.3355 +/- 0.4244
  Top-5 masking: confidence drop = 0.3592 +/- 0.4142
  Top-10 masking: confidence drop = 0.4938 +/- 0.4420
```

```
# Stability summary plot
fig, axes = plt.subplots(1, 3, figsize=(16, 5))

# SENS_MAX
eps_list = list(stability_results.keys())
axes[0].plot(eps_list, [stability_results[e]['sens_max'] for e in eps_list], 'o-', color='steelblue', markersize=8)
axes[0].set_xlabel('Perturbation epsilon'); axes[0].set_ylabel('SENS_MAX')
axes[0].set_title('SHAP Sensitivity (lower = more stable)'); axes[0].grid(alpha=0.3)

# PCC
pcc_vals = [stability_results[e]['pcc'] for e in eps_list]
colors = ['green' if p > 0.6 else 'red' for p in pcc_vals]
axes[1].bar(range(len(eps_list)), pcc_vals, color=colors)
axes[1].set_xticks(range(len(eps_list))); axes[1].set_xticklabels([f'eps={e}' for e in eps_list])
axes[1].axhline(y=0.6, color='gray', linestyle='--', label='Threshold (0.6)')
axes[1].set_ylabel('Mean PCC'); axes[1].set_title('SHAP Stability'); axes[1].legend()

# Faithfulness
ks = list(faith_results.keys())
axes[2].bar(range(len(ks)), [np.mean(faith_results[k]) for k in ks],
            yerr=[np.std(faith_results[k]) for k in ks], color='coral', capsize=5)
axes[2].set_xticks(range(len(ks))); axes[2].set_xticklabels([f'Top-{k}' for k in ks])
axes[2].set_ylabel('Confidence drop'); axes[2].set_title('Faithfulness (higher = better)')

plt.suptitle('Explanation Stability Evaluation (SAFARI Framework)', fontsize=14)
plt.tight_layout(); plt.show()
```



7. Security Implications Summary

```
# Analyze which top SHAP features are attacker-manipulable
manipulable = {'src_bytes', 'dst_bytes', 'hot', 'num_failed_logins', 'duration', 'num_compromised',
               'root_shell', 'su_attempted', 'num_root', 'num_file_creations', 'num_shells', 'num_access_files'}
partial = {'count', 'srv_count', 'serror_rate', 'srv_serror_rate', 'rerror_rate', 'srv_rerror_rate',
           'protocol_type', 'flag', 'service'}
non_manip = {'dst_host_count', 'dst_host_srv_count', 'dst_host_same_srv_rate', 'dst_host_diff_srv_rate',
             'dst_host_same_src_port_rate', 'dst_host_srv_diff_host_rate', 'dst_host_serror_rate',
             'dst_host_srv_serror_rate', 'dst_host_rerror_rate', 'dst_host_srv_rerror_rate',
             'same_srv_rate', 'diff_srv_rate', 'srv_diff_host_rate'}

print('SECURITY ANALYSIS: Top 15 Features by Manipulability')
print('='*70)
manip_count = {'Manipulable': 0, 'Partial': 0, 'Non-manipulable': 0}
for i, (f, v) in enumerate(feature_importance[:15]):
    if f in manipulable:
        status = 'MANIPULABLE'
        manip_count['Manipulable'] += 1
    elif f in partial:
        status = 'PARTIAL'
        manip_count['Partial'] += 1
    else:
        status = 'NON-MANIPULABLE'
        manip_count['Non-manipulable'] += 1
    print(f'  {i+1:2d}. {f:35s} SHAP={v:.4f}  [{status}]')

print(f'\nSummary: {manip_count}')
if manip_count['Non-manipulable'] > manip_count['Manipulable']:
    print('-> Model relies more on non-manipulable features -> MORE ROBUST against evasion')
else:
    print('-> Model relies more on manipulable features -> LESS ROBUST against evasion')
```

```
SECURITY ANALYSIS: Top 15 Features by Manipulability
======================================================================
   1. logged_in                           SHAP=0.0950  [NON-MANIPULABLE]
   2. dst_host_rerror_rate                SHAP=0.0619  [NON-MANIPULABLE]
   3. protocol_type                       SHAP=0.0573  [PARTIAL]
   4. rerror_rate                         SHAP=0.0479  [PARTIAL]
   5. dst_host_serror_rate                SHAP=0.0427  [NON-MANIPULABLE]
   6. count                               SHAP=0.0398  [PARTIAL]
   7. serror_rate                         SHAP=0.0380  [PARTIAL]
   8. dst_host_same_srv_rate              SHAP=0.0319  [NON-MANIPULABLE]
   9. same_srv_rate                       SHAP=0.0256  [NON-MANIPULABLE]
  10. dst_host_srv_serror_rate            SHAP=0.0252  [NON-MANIPULABLE]
  11. srv_rerror_rate                     SHAP=0.0231  [PARTIAL]
```